

1. Hosting a Local Web Server

To begin, I set up a simple local web server using Python's built-in `http.server` module. I first opened the terminal and created a dedicated directory for this project called `pki_lab` in my home folder. This helped keep all files and network traces organized in one location. Inside the directory, I added a basic HTML file called `index.html`, containing the line "Hello from HTTP" to serve as test content for my server.

Next, I started the server by running the following command:

```
python3 -m http.server 8080
```

```
The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
[Hanss-MacBook-Pro:~ hansamemiya$ cd ~
[Hanss-MacBook-Pro:~ hansamemiya$ mkdir pki_lab
[Hanss-MacBook-Pro:~ hansamemiya$ cd pki_lab
[Hanss-MacBook-Pro:pki_lab hansamemiya$ echo "Hello from HTTP!" > index.html
-bash: !": event not found
[Hanss-MacBook-Pro:pki_lab hansamemiya$ echo "Hello from HTTP" > index.html
[Hanss-MacBook-Pro:pki_lab hansamemiya$ python3 -m http.server 8080
Serving HTTP on :: port 8080 (http://[::]:8080/) ...
::1 - - [20/Oct/2025 10:09:31] "GET / HTTP/1.1" 200 -
::1 - - [20/Oct/2025 10:09:32] code 404, message File not found
::1 - - [20/Oct/2025 10:09:32] "GET /favicon.ico HTTP/1.1" 404 -
::1 - - [20/Oct/2025 10:09:35] "GET /index.html HTTP/1.1" 200 -
```

This command launched an HTTP server bound to port 8080 and served all files in the current working directory. Once the server was running, I opened a web browser and navigated to `http://localhost:8080`. The page loaded successfully, displaying my test message, confirming that the server was functioning properly.



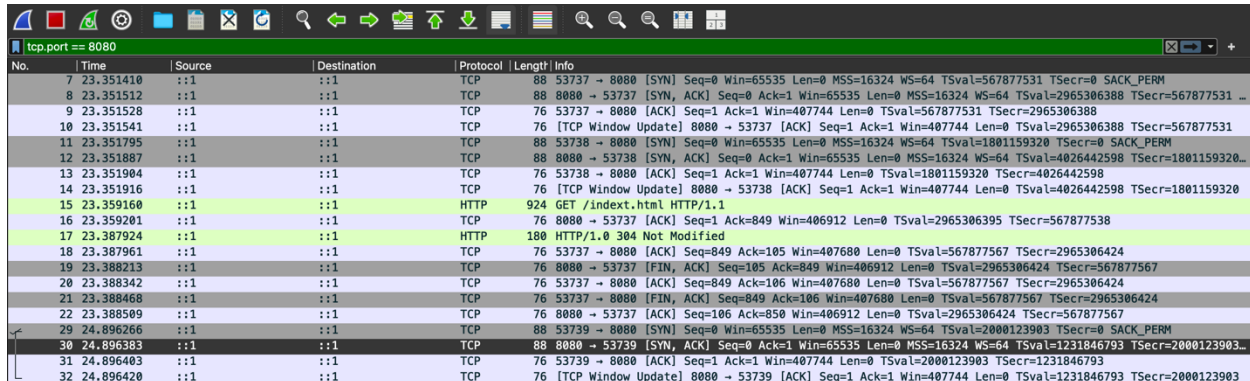
2. Why HTTP Is Not Secure

To demonstrate the insecurity of HTTP, I used Wireshark to capture packets exchanged between my local browser and my Python-based web server running on port 8080. I applied the filter `tcp.port == 8080` to isolate only the traffic related to the HTTP connection.

In the captured packets, I observed the client's request line:

GET /index.html HTTP/1.1

This line shows that the browser requested the file index.html from the local server using the HTTP protocol. The accompanying headers were also visible in plain text, as shown below.

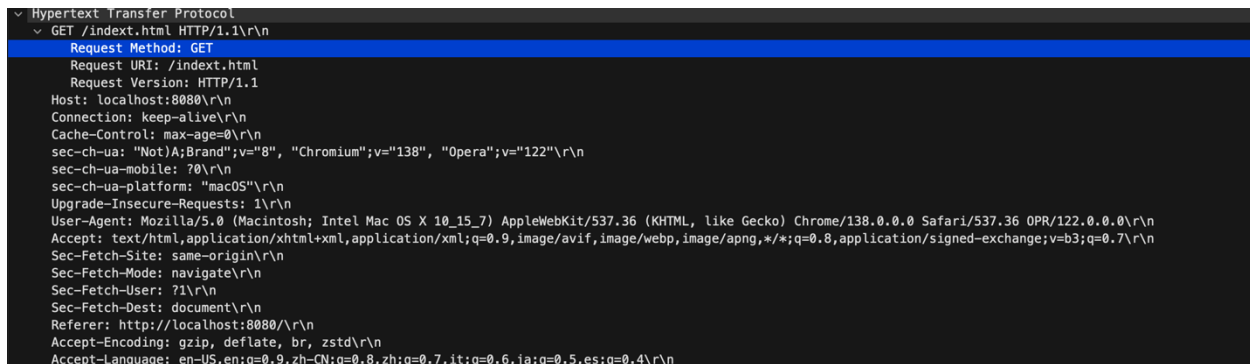


The image shows a Wireshark packet capture of a network session. The top bar indicates the filter is 'tcp.port == 8080'. The packet list on the left shows a series of TCP and HTTP packets. Packet 15 is highlighted, showing a GET request for /index.html. The packet details pane on the right shows the structure of the HTTP request, including the method, URI, version, host, and various headers.

No.	Time	Source	Destination	Protocol	Length	Info
7	23.351410	::1	::1	TCP	88	53737 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=16324 WS=64 TSval=567877531 TSecr=0 SACK_PERM
8	23.351512	::1	::1	TCP	88	8080 → 53737 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16324 WS=64 TSval=2965306388 TSecr=567877531 ..
9	23.351528	::1	::1	TCP	76	53737 → 8080 [ACK] Seq=1 Ack=1 Win=407744 Len=0 TSval=567877531 TSecr=2965306388
10	23.351541	::1	::1	TCP	76	[TCP Window Update] 8080 → 53737 [ACK] Seq=1 Ack=1 Win=407744 Len=0 TSval=2965306388 TSecr=567877531
11	23.351795	::1	::1	TCP	88	53738 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=16324 WS=64 TSval=1801159320 TSecr=0 SACK_PERM
12	23.351887	::1	::1	TCP	88	8080 → 53738 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16324 WS=64 TSval=4026442598 TSecr=1801159320..
13	23.351984	::1	::1	TCP	76	53738 → 8080 [ACK] Seq=1 Ack=1 Win=407744 Len=0 TSval=1801159320 TSecr=4026442598
14	23.351916	::1	::1	TCP	76	[TCP Window Update] 8080 → 53738 [ACK] Seq=1 Ack=1 Win=407744 Len=0 TSval=4026442598 TSecr=1801159320
15	23.359160	::1	::1	HTTP	924	GET /index.html HTTP/1.1
16	23.359281	::1	::1	TCP	76	8080 → 53737 [ACK] Seq=1 Ack=849 Win=406912 Len=0 TSval=2965306395 TSecr=567877538
17	23.387924	::1	::1	HTTP	180	HTTP/1.0 304 Not Modified
18	23.387961	::1	::1	TCP	76	53737 → 8080 [ACK] Seq=849 Ack=105 Win=407680 Len=0 TSval=567877567 TSecr=2965306424
19	23.388213	::1	::1	TCP	76	8080 → 53737 [FIN, ACK] Seq=105 Ack=849 Win=406912 Len=0 TSval=2965306424 TSecr=567877567
20	23.388342	::1	::1	TCP	76	53737 → 8080 [ACK] Seq=849 Ack=106 Win=407680 Len=0 TSval=567877567 TSecr=2965306424
21	23.388468	::1	::1	TCP	76	53737 → 8080 [FIN, ACK] Seq=849 Ack=106 Win=407680 Len=0 TSval=567877567 TSecr=2965306424
22	23.388599	::1	::1	TCP	76	8080 → 53737 [ACK] Seq=106 Ack=850 Win=406912 Len=0 TSval=2965306424 TSecr=567877567
29	24.896266	::1	::1	TCP	88	53739 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=16324 WS=64 TSval=2000123903 TSecr=0 SACK_PERM
30	24.896383	::1	::1	TCP	88	8080 → 53739 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16324 WS=64 TSval=1231846793 TSecr=2000123903..
31	24.896403	::1	::1	TCP	76	53739 → 8080 [ACK] Seq=1 Ack=1 Win=407744 Len=0 TSval=2000123903 TSecr=1231846793
32	24.896420	::1	::1	TCP	76	[TCP Window Update] 8080 → 53739 [ACK] Seq=1 Ack=1 Win=407744 Len=0 TSval=1231846793 TSecr=2000123903

They include:

- **Host:** localhost:8080 → the target server and port
- **User-Agent:** Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) → identifying the browser and operating system
- **Accept:** text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 → specifying acceptable content types
- **Referer:** http://localhost:8080/ → revealing the page that initiated the request



The image shows a detailed view of the HTTP request headers from the packet capture. The headers are listed in a structured format, showing the method, URI, version, host, and various headers like Cache-Control, sec-ch-ua, sec-ch-ua-mobile, sec-ch-ua-platform, Upgrade-Insecure-Requests, User-Agent, Accept, Sec-Fetch-Site, Sec-Fetch-Mode, Sec-Fetch-User, Sec-Fetch-Dest, Referer, Accept-Encoding, and Accept-Language.

```
Hypertext Transfer Protocol
  GET /index.html HTTP/1.1\r\n
    Request Method: GET
    Request URI: /index.html
    Request Version: HTTP/1.1
    Host: localhost:8080\r\n
    Connection: keep-alive\r\n
    Cache-Control: max-age=0\r\n
    sec-ch-ua: "Not)A;Brand";v="8", "Chromium";v="138", "Opera";v="122"\r\n
    sec-ch-ua-mobile: ?0\r\n
    sec-ch-ua-platform: "macOS"\r\n
    Upgrade-Insecure-Requests: 1\r\n
    User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36 OPR/122.0.0.0\r\n
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7\r\n
    Sec-Fetch-Site: same-origin\r\n
    Sec-Fetch-Mode: navigate\r\n
    Sec-Fetch-User: ?1\r\n
    Sec-Fetch-Dest: document\r\n
    Referer: http://localhost:8080/\r\n
    Accept-Encoding: gzip, deflate, br, zstd\r\n
    Accept-Language: en-US,en;q=0.9,zh-CN;q=0.8,zh;q=0.7,it;q=0.6,ja;q=0.5,es;q=0.4\r\n
```

Every field is transmitted unencrypted, meaning any observer intercepting the network packets could easily read this information. In the same trace, the server's response headers were also visible, for example:

HTTP/1.0 304 Not Modified

This indicates that the browser already had a cached version of the page, but even this status line and related metadata were fully legible.

The key security flaw here is that HTTP traffic is transmitted in plaintext. If this were a real network and not just localhost, any device between the client and server could intercept the

packets and see: which web pages are being accessed (/index.html), which browser and OS are used (User-Agent) and any data submitted via forms or URLs (GET /login?username=...)

3. HTTPS Network Trace and Analysis

(a) Why you can't obtain an SSL certificate from a public Certificate Authority for your local web server

Public Certificate Authorities only issue certificates for domain names that are publicly registered and for which the applicant can prove ownership or control. The hostname *localhost* and the IP address *127.0.0.1* are universal loopback identifiers used by every device, not unique to any one organization. Because they are not publicly registered or verifiable, no CA can confirm ownership. As a result, major providers explicitly refuse to issue certificates for private IP addresses. For local development environments, the only option is to create and use a self-signed certificate, which provides encryption but must be manually trusted on the local machine.

(b)

First, I created a short Python script called *serve_https.py* using the terminal and the nano text editor. This script launched a simple HTTPS server by loading the self-signed certificate (localhost.crt) and private key (localhost.key). Running the server hosted my page securely at <https://127.0.0.1:8443>.

```
UW PICO 5.09 File: serve_https.py

from http.server import HTTPServer, SimpleHTTPRequestHandler
import ssl

PORT = 8443

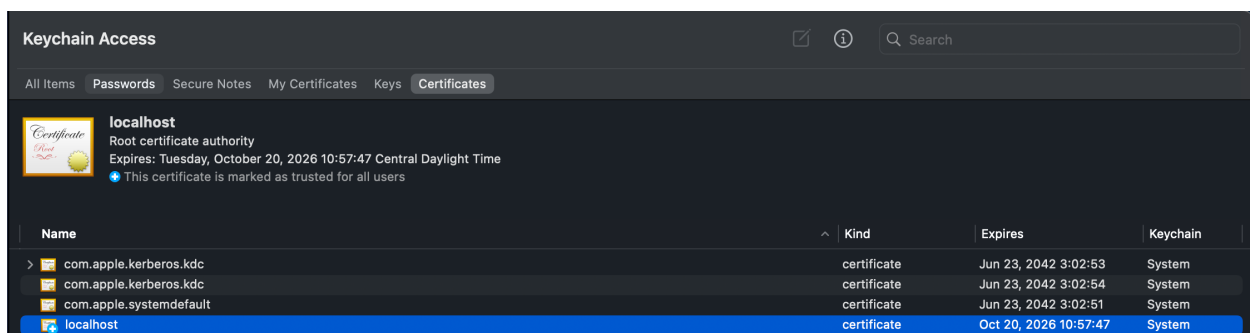
httpd = HTTPServer(("127.0.0.1", PORT), SimpleHTTPRequestHandler)

context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
context.load_cert_chain(certfile="localhost.crt", keyfile="localhost.key")

httpd.socket = context.wrap_socket(httpd.socket, server_side=True)

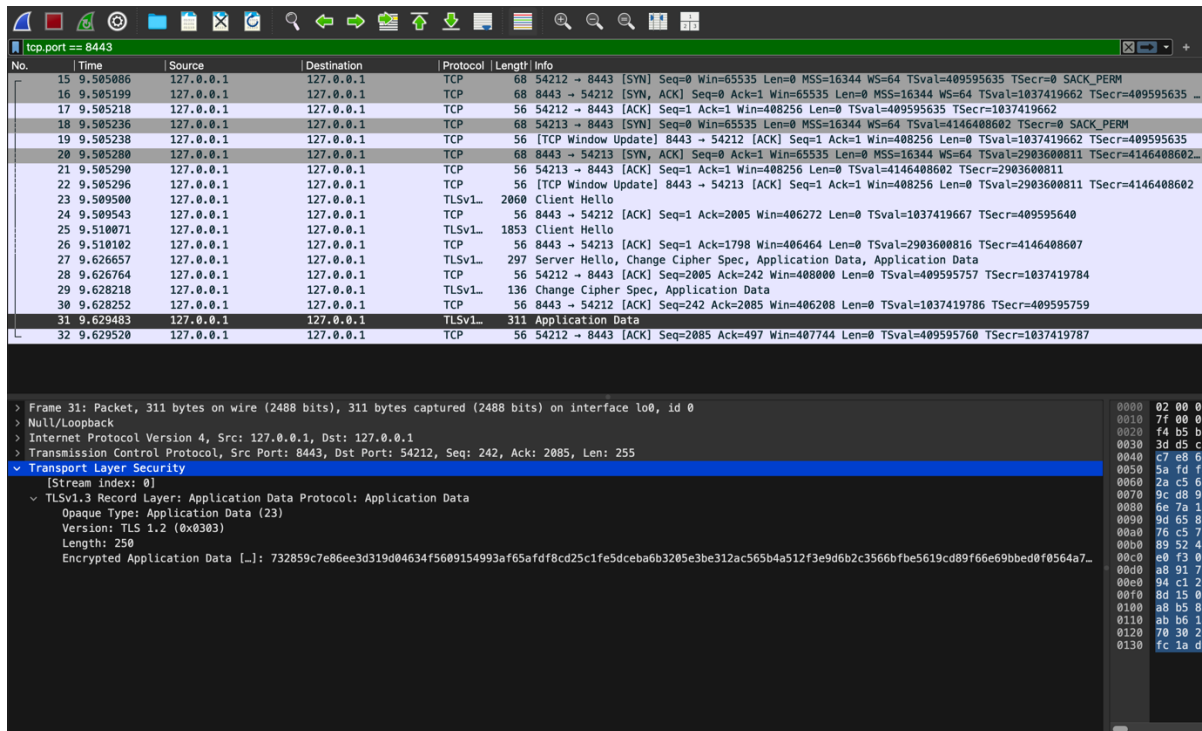
print(f"Serving HTTPS on https://127.0.0.1:{PORT}")
httpd.serve_forever()
```

Second, I added the certificate to Keychain Access under the *System keychain* to mark it as trusted. This was done by importing localhost.crt and setting it to “Always Trust.” Once added, the certificate appeared as “localhost” in Keychain Access, confirming it was trusted for all users.



After creating a self-signed certificate and upgrading my local web server to HTTPS, I accessed the page using `https://127.0.0.1:8443` and captured the traffic in Wireshark using the filter `tcp.port == 8443`.

The capture shows the TLS handshake followed by encrypted communication. The client (browser) first sends a “Client Hello,” and the server replies with a “Server Hello” and its certificate. Unlike my earlier HTTP capture where the request line (`GET /index.html`) and headers were clearly readable, the HTTPS packets only contain encrypted data. For example, one of the packets in the trace shows an “Encrypted Application Data” field with random hexadecimal values instead of text.



The image shows a Wireshark packet capture of a TLS handshake and encrypted application data. The filter is set to `tcp.port == 8443`. The packet list shows the following details:

No.	Time	Source	Destination	Protocol	Length	Info
15	9.585886	127.0.0.1	127.0.0.1	TCP	68	54212 → 8443 [SYN] Seq=0 Win=65535 Len=0 MSS=16344 WS=64 TSval=409595635 TSecr=0 SACK_PERM
16	9.585199	127.0.0.1	127.0.0.1	TCP	68	8443 → 54212 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=1037419662 TSecr=409595635
17	9.585218	127.0.0.1	127.0.0.1	TCP	56	54212 → 8443 [ACK] Seq=1 Ack=1 Win=408256 Len=0 TSval=409595635 TSecr=1037419662
18	9.585236	127.0.0.1	127.0.0.1	TCP	68	54213 → 8443 [SYN] Seq=0 Win=65535 Len=0 MSS=16344 WS=64 TSval=4146408602 TSecr=0 SACK_PERM
19	9.585238	127.0.0.1	127.0.0.1	TCP	56	[TCP Window Update] 8443 → 54212 [ACK] Seq=1 Ack=1 Win=408256 Len=0 TSval=1037419662 TSecr=409595635
20	9.585280	127.0.0.1	127.0.0.1	TCP	68	8443 → 54213 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=2903600811 TSecr=4146408602
21	9.585290	127.0.0.1	127.0.0.1	TCP	56	54213 → 8443 [ACK] Seq=1 Ack=1 Win=408256 Len=0 TSval=4146408602 TSecr=2903600811
22	9.585296	127.0.0.1	127.0.0.1	TCP	56	[TCP Window Update] 8443 → 54213 [ACK] Seq=1 Ack=1 Win=408256 Len=0 TSval=2903600811 TSecr=4146408602
23	9.589500	127.0.0.1	127.0.0.1	TLSv1..	2060	Client Hello
24	9.589543	127.0.0.1	127.0.0.1	TCP	56	8443 → 54212 [ACK] Seq=1 Ack=2085 Win=406272 Len=0 TSval=1037419667 TSecr=409595640
25	9.510071	127.0.0.1	127.0.0.1	TLSv1..	1853	Client Hello
26	9.510102	127.0.0.1	127.0.0.1	TCP	56	8443 → 54213 [ACK] Seq=1 Ack=1798 Win=406464 Len=0 TSval=2903600816 TSecr=4146408607
27	9.626657	127.0.0.1	127.0.0.1	TLSv1..	297	Server Hello, Change Cipher Spec, Application Data, Application Data
28	9.626764	127.0.0.1	127.0.0.1	TCP	56	54212 → 8443 [ACK] Seq=2005 Ack=242 Win=408000 Len=0 TSval=409595757 TSecr=1037419784
29	9.628218	127.0.0.1	127.0.0.1	TLSv1..	136	Change Cipher Spec, Application Data
30	9.628252	127.0.0.1	127.0.0.1	TCP	56	8443 → 54212 [ACK] Seq=242 Ack=2085 Win=406208 Len=0 TSval=1037419786 TSecr=409595759
31	9.629483	127.0.0.1	127.0.0.1	TLSv1..	311	Application Data
32	9.629520	127.0.0.1	127.0.0.1	TCP	56	54212 → 8443 [ACK] Seq=2085 Ack=497 Win=407744 Len=0 TSval=409595760 TSecr=1037419787

The packet details for Frame 31 (Application Data) show:

- Stream Index: 0
- TLSv1.3 Record Layer: Application Data Protocol: Application Data
- Opaque Type: Application Data (23)
- Version: TLS 1.2 (0x0303)
- Length: 250
- Encrypted Application Data [...]: 732859c7e86ee3d319d04634f5609154993af65afdf8c2d5c1fe5dceba6b3205e3be312ac565b4a512f3e9d6b2c3566bfbe5619cd89f66e69bbd0f0564a7...

The packet bytes pane shows the encrypted data as a series of hexadecimal values:

```
0000 02 00 00
0010 7f 00 00
0020 f4 05 bc
0030 3d 05 c5
0040 c7 e8 6e
0050 5a fd f8
0060 2a c5 65
0070 9c 08 9f
0080 6e 7a 13
0090 9d 65 8f
00a0 76 c5 75
00b0 89 52 48
00c0 e0 f3 02
00d0 a8 91 7f
00e0 94 c1 26
00f0 8d 15 03
0100 a8 b5 8a
0110 ab b6 18
0120 70 30 26
0130 fc 1a d9
```

In the browser, this change is visually reflected by the appearance of a lock icon next to the URL, indicating that the connection is now secure. This confirms that HTTPS not only encrypts requests and responses but also authenticates the server and protects against tampering.



This confirms that HTTPS protects communication by encrypting both requests and responses. An outside observer could still see that a connection occurred and the size of each packet, but not the content or specific pages accessed.